راهنمای کد نویسی و استفاده از SMS_BAN

فهرست مطالب

شیوه رجیستر کردن ACTIVEX ۱

- رجیستر کردن ActiveX در ویندوز ۳۲ بیتی ۱
- رجیستر کردن ActiveX در ویندوز ٦٤ بیتی ۳

اضافه کردن کنترل ACTIVEX به محیط برنامه نویسی در زبان های مایکروسافت مثال در C# ٤

- اضافه کردن کنترل ActiveX روی فرم ٤
- اضافه کردن کلاس ActiveX به پروژه و استفاده از Object آن ٥
- اضافه کردن کلاس ActiveX به یک پروژه ی DLL و استفاده از DLL در پروژه ی اصلی ٦

اضافه کردن کنترل ACTIVEX در محیط برنامه نویسی DELPHI ۷

معرفی توابع ACTIVEX ۸

- توابع اولیه برای انتخاب دستگاه ۸
  - تابع شناسایی دستگاههای متصل ۸
  - تابع انتخاب دستگاه ۹
  - تابع نمایش نسخه سخت افزار ۹
  - تابع نمایش نسخه نرم افزار ۱۰
  - تابع تست اتصال مودم ۱۱
  - تابع آماده بودن مودم ۱۱
  - تابع تنظیم نوع پیام ارسالی ۱۲
  - تابع تنظیم تاریخ و ساعت ۱۳
  - تابع خواندن تاریخ و زمان ۱٤

- تابع پاک کردن پیامک ١٤
- تابع خواندن تعداد پیامک های رسیده ١٥
- تابع خواندن شدت آنتن دهی سیم کارت ١٦
- تابع تنظیم نوع کاراکتر پیامک ١٦
- تابع خواندن پیامک رسیده ١٧
- تابع دوم خواندن پیامک رسیده ١٩
- تابع سوم خواندن پیامک رسیده ٢١
- تابع ارسال پیامک ٢٤
- تابع دوم ارسال پیامک ٢٦
- تابع خواندن میزان شارژ باتری ٢٧
- تابع وضعیت شارژ شدن باتری ٢٨
- تابع برقرای تماس ٢٩
- تابع ارسال پیامک **Text** ٣٠
- تابع ارسال ارسال **AT Command** ٣١
- تابع ارسال ارسال **USSD** ٣٢
- تابع ارسال دریافت **USSD** ٣٣
- تابع تنظیم شماره دستگاه ٣٣
- تابع ارسال ارسال **AT Command** ٣٤

مقدمه

SMS_BAN دستگاهی است که برای ارسال و دریافت پیامک از طریق سیم کارت مورد استفاده قرار می گیرد.

SMS_BAN به همراه خود یک اکتیوایکس دارد که برنامه نویس می تواند با استفاده از آن و از طریق درگاه USB با دستگاه مرتبط شود.

برای برقراری ارتباط میان نرم افزار و دستگاه SMS_BAN ابتدا باید فایل ActiveX دستگاه (MSPOSBAN.ocx) روی کامپیوتر برنامه نویس یا مشتری رجیستر شود.

چنانچه برنامه نویس یا شرکت مایل نباشد روی سیستم کاربر نهایی شان فایل ActiveX رجیستر شود باید از Manifest استفاده کند.

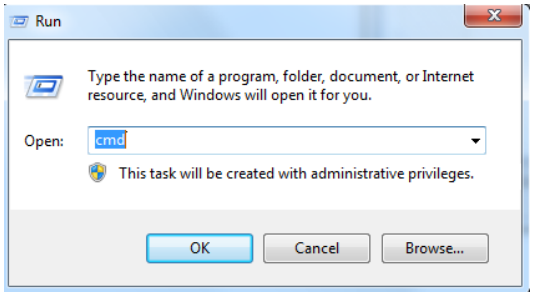
شیوه رجیستر کردن ACTIVEX

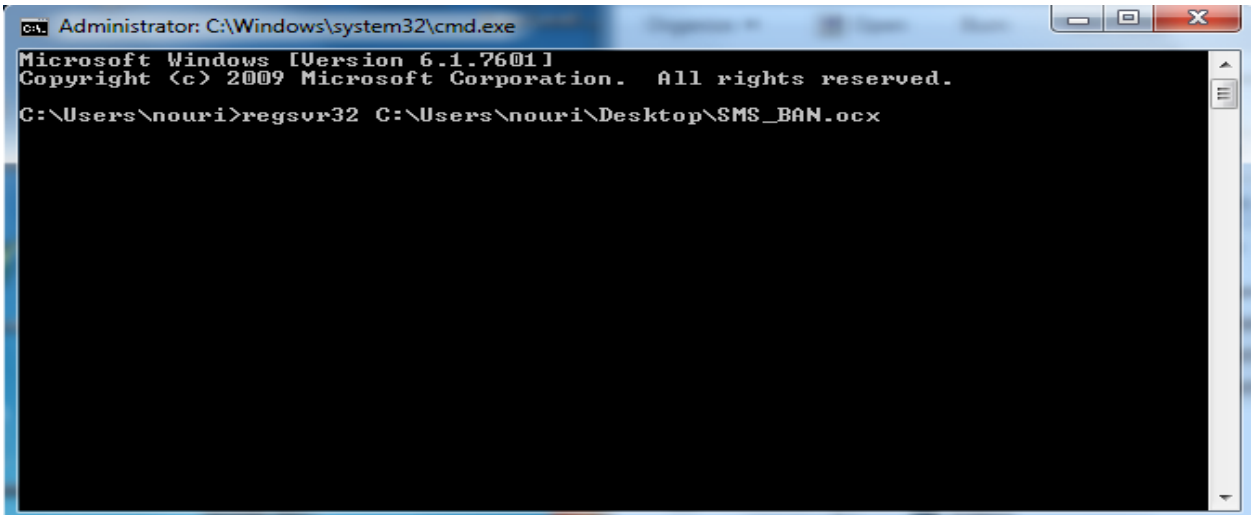
برای رجیستر کردن فایل ActiveX پیشنهاد می شود که Administrator ویندوز فعال باشد و در این کاربری ActiveX رجیستر شود اما گاهی، صرفاً با اجرای Command Prompt به صورت Run As Administrator مشکل حل می شود.

- **رجیستر کردن ActiveX در ویندوز ۳۲ بیتی**

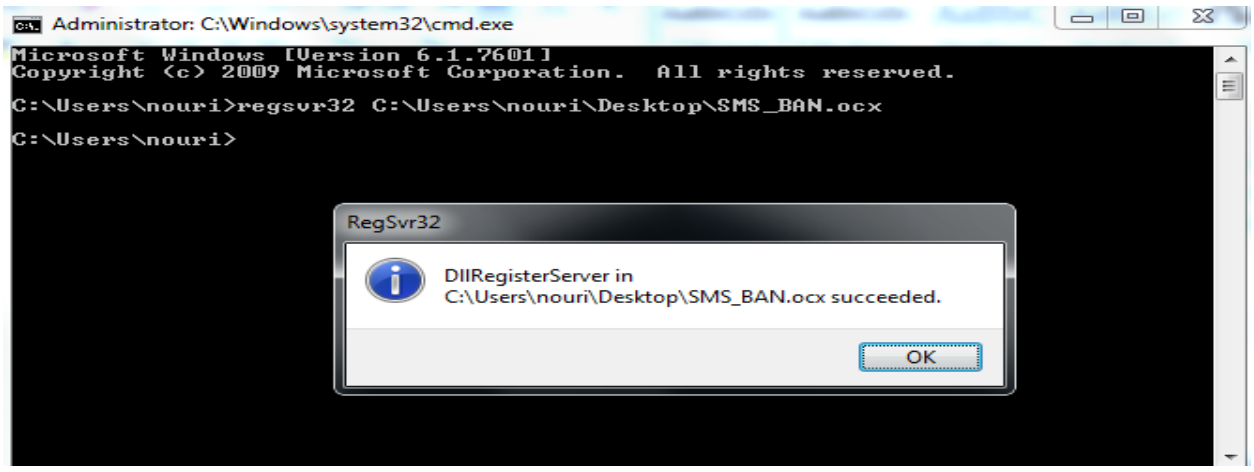
فایل SMS_BAN.ocx که همراه دستگاه بر روی CD قرار دارد، می بایست در مسیر ثابتی روی هارد سیستم قرار گیرد، پس از این عملیات Command Prompt اجرا شده و دستور زیر در آن تایپ شود:

نام فایل + مسیر کامل فایل Regsvr32





با اجرای دستور فوق باید ویندوز پیغام زیر را نمایش دهد.



- **رجیستر کردن ActiveX در ویندوز ۶۴ بیتی**

مسیر کامل فایل به همراه نام فایل Regsvr32

برای مثال :

Regsvr32 "D:\MyFolder\ SMS_BAN.ocx"

با اجرای دستور فوق باید ویندوز به شما پیغام succeeded دهد،در غیر این صورت ، روش زیر را تست کنید :

فایل SMS_BAN.ocx را در مسیر C:\Windows\SysWOW64 کپی کنید و هنگام رجیستر کردن هم مسیر فوق را بدهید. در اکثر مواقع با این کار،رجیسترکردن ActiveX روی ویندوز۶۴ بیتی انجام می شود.

بعد از رجیستر کردن ActiveX بستر برنامه نویسی برای شما فراهم می شود.

نکات مهم:

***توجه کنید در مسیر فایل ActiveX نباید پوشه ای با نام فارسی قرار گرفته باشد.

*** اگر در انتهای مسیر فایل بعد از کاراکتر فضای خالی /u بگذاریم اکتیو ایکس Unregister می شود و اگر /s بگذاریم پیغام پس از Register یا Un Register نمایش داده نخواهد شد.

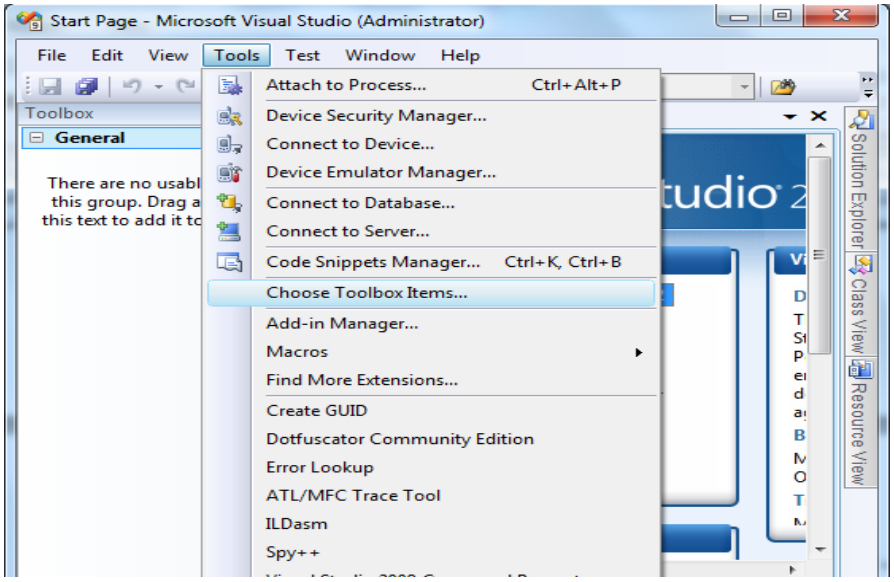
اضافه کردن کنترل ACTIVEX به محیط برنامه نویسی در زبان های مایکروسافت مثال در C#

حال با توجه به محیطی که در آن برنامه نویسی می کنید باید کنترلActiveX را به پروژه ی خود اضافه کنید.

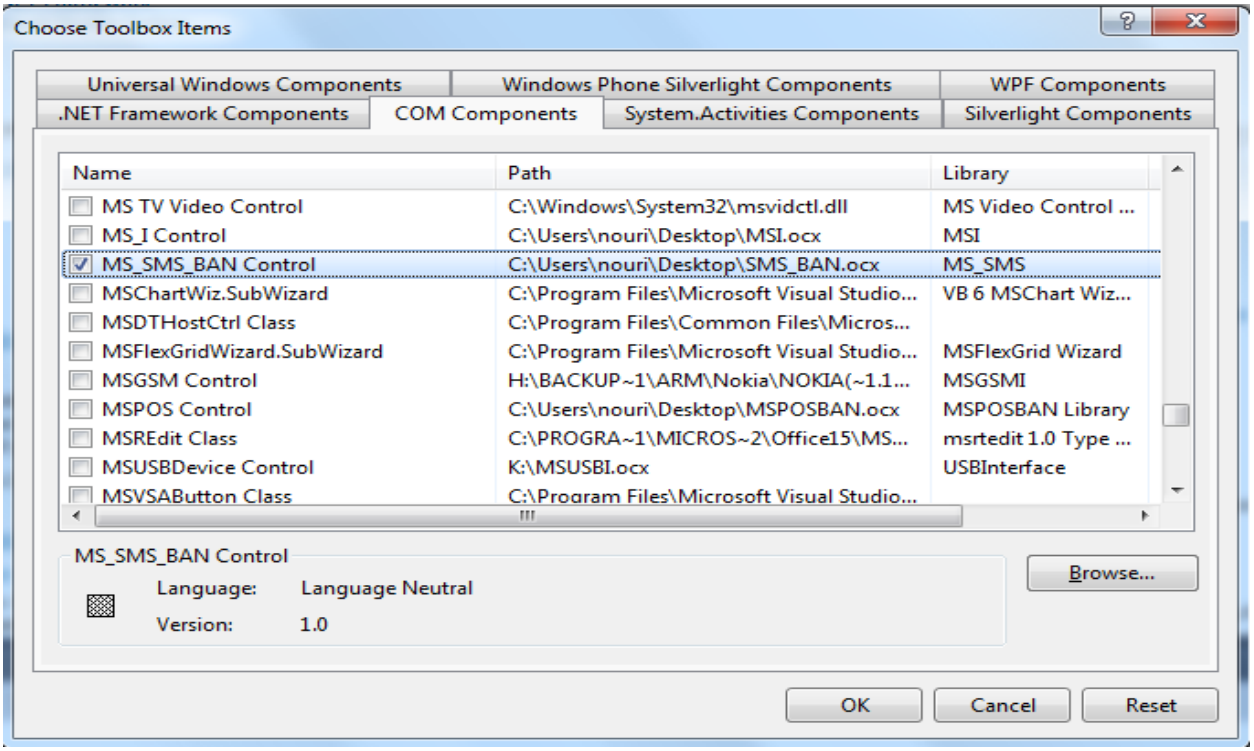
مثلا در C# این کار بدین صورت انجام می شود :

- **اضافه کردن کنترل ActiveX روی فرم**

از منوی Tools گزینه "Choose Toolbox Items..." را انتخاب کنید به صورت زیر :



از پنجره ی باز شده در زبانه ، COM Component گزینه ی MS_SMS_BAN Control را تیک بزنید و OK کنید(همانند شکل زیر).



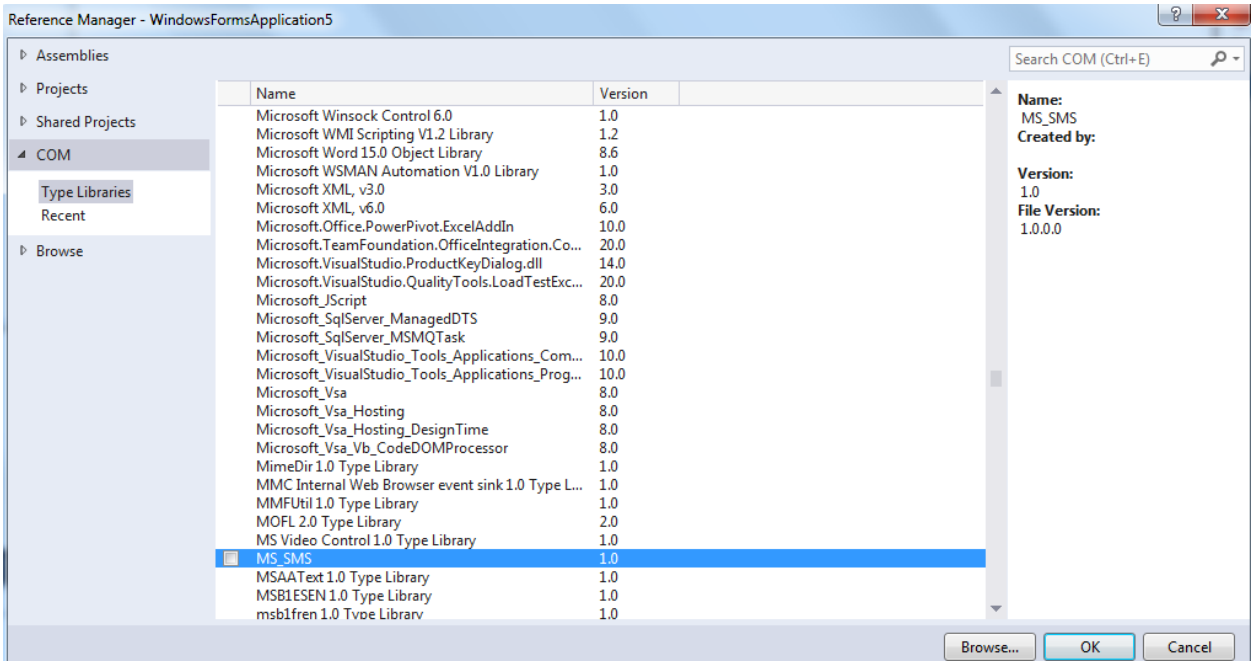
با این کار کنترل MS_SMS_BAN Control به Tool Box شما اضافه می شود و شما می توانید با اضافه کردن کنترل ،به فرم خود،از توابع از پیش تعریف شده برای شناسایی قفل استفاده کنید که شیوه ی کار با این توابع در زیر توضیح داده شده است.

- **اضافه کردن کلاس ActiveX به پروژه و استفاده از Object آن**

اما چنانچه مایل نبودید کنترل را به فرم خود اضافه کنید و فقط می خواستید از کلاس آن یک Object بسازید و با آن کار کنید به صورت زیر اقدام می کنید :

در محیط Visual Studio در پنجره Solution Explorer کلیک راست کرده و از منوی باز شده گزینه Add→Reference را انتخاب نمایید.

در پنجره باز شده گزینه ، Com را انتخاب کرده و از لیست، گزینه MS_SMS را انتخاب کنید.



پس از کلیک روی دکمه OK ماژول MS_SMS_BAN به پروژه اضافه می شود. پس از این مرحله می توانید با تعریف یک Object از این ماژول به توابع و متغییر های این ماژول دسترسی پیدا کنید.

- **اضافه کردن کلاس ActiveX به یک پروژه ی DLL و استفاده از DLL در پروژه ی اصلی**

چنانچه مایل بودید SMS_BAN را در یک dll استفاده کنید باید به شیوه ی زیر عمل کنید :

به پروژه ی dll خود یک Form اضافه کنید به فرم اضافه شده یک بار کنترل SMS_BAN را از Tool box اضافه کنید سپس آنرا پاک کنید،با این کار MS_SMS_BAN Ax به Reference های پروژه ی شما اضافه می شود . حال در کد های خود، در آن بخشی که می خواهید Object ای از کلاسSMS_BAN تعریف کنید به شیوه ی زیر عمل کنید :

```
string d_i;
Form1 frm = new Form1();
AxMS_SMS.AxMS_SMS_BAN  mssms;
mssms = new AxMS_SMS.AxMS_SMS_BAN();
mssms.CreateControl();
mssms.Parent = frm;

d_i = mssms.SMS_Dev_INFO();
```

ابتدا object را به صورت Global از کلاس AxMS_SMS_BAN تعریف می کنیم.

در سازنده ی کلاس آن را New می کنیم سپس تابع Create Control را صدا زده بعد برای آن Parent تعریف می کنیم که این Parent همان فرم خالی ما است.(Object ساخته شده از کلاس AxMS_SMS_BAN حتما باید Parent داشته باشد.)

سپس توابعی که در ذیل معرفی می شود جهت کار با دستگاه مورد استفاده قرار می گیرد .

اضافه کردن کنترل ACTIVEX در محیط برنامه نویسی DELPHI

۱. در دلفی یک Package ایجاد کنید.
۲. در Package ایجاد شده از منوی Component گزینه ی Import Component را انتخاب کنید
۳. در پنجره ی باز شده تیک Import ActiveX Control را بزنید و Next کنید.
۴. در کادر search عبارت SMS_BAN را تایپ کنید تا ActiveX ای که رجیستر کردید را برای شما پیدا کند.
۵. Next کنید.

۶. در این پنجره Package ای که ایجاد کردید را انتخاب کنید و دکمه ی finish را بزنید.با این کار ActiveX به این Package اضافه شده است، حال Package را Compile کنید و سپس Install کنید، با این کار کنترل ActiveX به لیست کنترل های شما اضافه خواهد شد و می توانید از ان استفاده کنید.

معرفی توابع ACTIVEX

- **توابع اولیه برای انتخاب دستگاه**

➢ **تابع شناسایی دستگاههای متصل**

SMS_Device_Count(): Smallint;

➢ **پارامترها** :

-

➢ **مقدار بازگشتی:**

عدد صحیح ← تعداد دستگاههای شناسایی شده

➢ **توضیحات:**

این تابع تعداد دستگاههای متصل به کامپیوتر و شناسایی شده را نمایش می دهد.

- **تابع انتخاب دستگاه**

SMS_Select_Device (Indx: smallint): WideString;

- **پارامترها :**

عدد صحیح ←شماره دستگاه که از ۱ شروع می شود.

- **مقدار بازگشتی:**

رشته ←نام برچسب دستگاه انتخاب شده

- **توضیحات:**

این تابع برای انتخاب دستگاه مورد نظر می باشد .

شماره دستگاه بر اساس آیتم های منوی کرکره ایی که در اکتیو ایکس نمایش داده شده است، می-باشد.

- **تابع نمایش نسخه سخت افزار**

SMS_Dev_INFO (): WideString;

- **پارامترها :**

-

- **مقدار بازگشتی:**

رشته ← نمایش نسخه سخت افزار

- **توضیحات:**

برای نمایش مشخصات و شماره نسخه دستگاه می باشد.

- **تابع نمایش نسخه نرم افزار**

SMS_Soft_Ver(): WideString;

- **پارامترها :**

-

- **مقدار بازگشتی:**

رشته ← نمایش نسخه نرم افزار

- **توضیحات:**

برای نمایش مشخصات و شماره نسخه اکتیوایکس می باشد.

- **تابع تست اتصال مودم**

GSM_Test_Connect(): WordBool;

- **پارامترها :**

-

- **مقدار بازگشتی:**

عدد منطقی ← True به این معنی آن است که نرم افزار به GSM متصل است و False به معنای عدم ارتباط با GSM است.

- **توضیحات:**

پس از روشن شدن و بارگذاری مودم ، ارتباط با مودم برقرار می گردد، این تابع وضعیت برقراری ارتباط با GSM را نمایش می دهد.

- **تابع آماده بودن مودم**

GSM_Ready(): WordBool;

- **پارامترها :**

-

- **مقدار بازگشتی:**

عدد منطقی ← True به این معنی آن است که مودم آماده کار است و False به معنای عدم آماده بودن مودم است.

- **توضیحات**:

پس از روشن شدن و بارگذاری مودم، ارتباط سیم کارت با مراکز مخابراتی برقرار می شود ، با استفاده از این تابع آماده شدن سیم کارت و مودم جهت استفاده مشخص می شود.

- **تابع تنظیم نوع پیام ارسالی**

SMS_PDU_Mode (PDU: WordBool): WordBool;

- **پارامترها** :

PDU ← نوع پیام

- **مقدار بازگشتی:**

عدد منطقی ← اگر تنظیم بر روی مودم با موفقیت انجام شود مقدار برگشتی تابع True و در غیر اینصورت False خواهد بود.

- **توضیحات:**

دو نوع پیام کوتاه وجود دارد PDU یا نوع کاراکترهای بین المللی که فارسی را نیز شامل می شود و نوع دیگر که فقط پیامهای انگلیسی(هر کاراکتر یک بایت) را ارسال می کند.

- **تابع تنظیم تاریخ و ساعت**

Set_GSM_Date_Time(GSM_Year,GSM_Month, GSM_Day, GSM_Hour, GSM_Minute, GSM_Second: Smallint): WordBool;

- **پارامترها :**

GSM_Year ← سال

GSM_Month ← ماه

GSM_Day ← روز

GSM_Hour ← ساعت

GSM_Minute ← دقیقه

GSM_Second ← ثانیه

- **مقدار بازگشتی:**

عدد منطقی ← اگر تنظیم ساعت در مودم با موفقیت انجام شود خروجی این تابع، True خواهد بود، در غیر اینصورت False خواهد بود.

- **توضیحات:**

از این تابع برای تنظیم ساعت مودم استفاده می شود.

- **تابع خواندن تاریخ و زمان**

R_GSM_Date_Time(): WideString;

- **پارامترها :**

-

- **مقدار بازگشتی:**

رشته ← خروجی این تابع تاریخ و ساعت به ترتیب: سال، ماه، روز، ساعت، دقیقه و ثانیه می باشد.

- **توضیحات:**

از این تابع برای خواندن تاریخ و ساعت مودم استفاده می شود.

نمونه خروجی:D:17/10/20 T:12:5:20

- **تابع پاک کردن پیامک**

SMS_Delete(Indx, Del_Type: Integer): WordBool;

- **پارامترها :**

Indx ← شماره پیامک جهت حذف

Del_Type ← نوع حذف پیامک

مقادیر Del_Type:

- ۰: با شماره پیامک.
- ۱: پیام خوانده شده.
- ۲: پیامک خوانده و ارسال شده.
- ۳: خوانده،ارسال شده و ارسال نشده.
- ۴: تمام پیام.

کاربرد Indx:

- اگر پاک کردن پیامک بر اساس شماره پیامک(Del_Type = صفر) باشد این پارامتر از مقدار ۱ تا تعداد پیامکهای رسیده، می تواند مقدار دهی گردد.

➢ **مقدار بازگشتی:**

عدد منطقی ← اگر حذف پیامک در مودم با موفقیت انجام شود خروجی این تابع، True خواهد بود، در غیر اینصورت False خواهد بود.

➢ **توضیحات:**

از این تابع جهت حذف پیامک های رسیده استفاده می شود.

➢ **تابع خواندن تعداد پیامک های رسیده**

Get_SMS_NO(): Integer;

➢ **پارامترها :**

-

- **مقدار بازگشتی:**

عدد صحیح ← اگر ارتباط با مودم برقرار شود تعداد پیامک های رسیده را نمایش می دهد در غیر اینصورت ۱- بر می گرداند.

- **توضیحات:**

این تابع برای خواندن تعداد پیامک های رسیده به سیم کارت مورد استفاده قرار می گیرد.

- **تابع خواندن شدت آنتن دهی سیم کارت**

```
GSM_Signal():WideString;
```

- **پارامترها :**

-

- **مقدار بازگشتی:**

رشته ← خروجی این تابع، شدت آنتن دهی به درصد می باشد.

اگر سیگنال نداشته باشد رشته NO Signal را بر می گرداند.

- **توضیحات:**

جهت خواندن میزان آنتن دهی سیم کارت از صفر درصد تا ۱۰۰ درصد مورد استفاده قرار می گیرد.

- **تابع تنظیم نوع کاراکتر پیامک**

SMS_Char_Mode(Char_Mode: Smallint):WordBool;

- **پارامترها :**

- **Char_Mode** ← نوع کاراکتر پیامک که به ترتیب از اندیس ۱ تا ۵ عبارتست از:

  - GSM
  - HEX
  - IRA
  - PCCP437
  - 8859-1
  - UCS2

- **مقدار بازگشتی:**

عدد منطقی ← در صورتی که تنظیم با موفقیت انجام شود مقدار True و در غیر اینصورت مقدار False بر می گرداند.

- **توضیحات:**

برای تنظیم نوع کاراکتر ارسالی پیامک مورد استفاده قرار می گیرد.

نکته۱: برای تغییر نوع ارسال پیامک از PDU به Text ابتدا باید نوع کاراکتر را GSM تنظیم نمود و سپس نوع پیامک Text تنظیم شود.

نکته۲: برای تغییر نوع ارسال پیامک از Text به PDU ابتدا باید نوع کاراکتر را UCS2 تنظیم نمود و سپس نوع پیامک PDU تنظیم شود.

- **تابع خواندن پیامک رسیده**

R_SMS(PDU: WordBool; Indx: Integer; var SMS_Center, SMS_Type, Tel_Number,SMS_Date_Time, SMS_Text: WideString): WordBool;

- **پارامترها :**
- **PDU** ← جهت خواندن نوع پیامک (PDU یا Text) مورد استفاده قرار می گیر و در حال حاضر فقط True می باشد.
  - نوع پارامتر ورودی است.

Indx ← شماره پیامک

  - نوع پارامتر ورودی است

SMS_Center←شماره مرکز پیام ، پیامک خوانده شده.

  - نوع پارامتر خروجی است

SMS_Type← نوع پیامک رسیده که شامل موارد ذیل می باشد:

- ✓ SMS RECEIVED: پیامک رسیده
- ✓ SMS STATUS REPORT: تاییدیه پیامک(Delivery)
- ✓ SMS SUBMIT: پیامک از تلفن همراه به مرکز پیام
- ✓ EMS RECEIVED(خدمات پیام رسانی پیشرفته): پیامک پیشرفته دریافت شده
- ✓ EMS SUBMIT: پیامک پیشرفته از تلفن همراه به مرکز پیام
  - نوع پارامتر خروجی است

Tel_Number← شماره تلفن پیامک ارسالی

- نوع پارامتر خروجی است

SMS_Date_Time ← تاریخ و ساعت ارسال پیامک

- نوع پارامتر خروجی است

SMS_Text ← متن پیامک

- نوع پارامتر خروجی است

➢ **مقدار بازگشتی:**

عدد منطقی ← در صورتی که پیامک با موفقیت خوانده شود مقدار برگشتی True و در غیر اینصورت False خواهد بود.

➢ **توضیحات:**

برای خواندن پیامک رسیده از این تابع استفاده می شود.

➢ **تابع دوم خواندن پیامک رسیده**

R_SMS2(PDU: WordBool; Indx: Integer; var SMS_Center, SMS_Type, Tel_Number,SMS_Date_Time, SMS_Text: WideString): WideString;

➢ **پارامترها :**

PDU ← جهت خواندن نوع پیامک (PDU یا Text) مورد استفاده قرار می گیر و در حال حاضر فقط True می باشد.

- نوع پارامتر ورودی است

Indx ← شماره پیامک

- نوع پارامتر ورودی است

SMS_Center←شماره مرکز پیام ، پیامک خوانده شده.

- نوع پارامتر خروجی است

SMS_Type← نوع پیامک رسیده که شامل موارد ذیل می باشد:

- ✓ SMS RECEIVED: پیامک رسیده
- ✓ SMS STATUS REPORT: تاییدیه پیامک(Delivery)
- ✓ SMS SUBMIT: پیامک از تلفن همراه به مرکز پیام
- ✓ EMS RECEIVED(خدمات پیام رسانی پیشرفته): پیامک پیشرفته دریافت شده
- ✓ EMS SUBMIT: پیامک پیشرفته از تلفن همراه به مرکز پیام

- نوع پارامتر خروجی است

Tel_Number← شماره تلفن پیامک ارسالی

- نوع پارامتر خروجی است

SMS_Date_Time← تاریخ و ساعت ارسال پیامک

- نوع پارامتر خروجی است

SMS_Text← متن پیامک

- نوع پارامتر خروجی است

➢ **مقدار بازگشتی:**

رشته کاراکتری ← در صورتی که پیامک با موفقیت خوانده شود مقدار برگشتی یک رشته شامل یکسری کاراکتر عددی می باشد که هر چهار رقم آن با ویرگول از هم جدا شدند این اعداد چهار رقمی کد دوبایتی یک کاراکتر پیامک می باشد ، به عنوان مثال عدد هگز 0X06F1 کد دو کاراکتری عدد ۱ فارسی است.

➢ **توضیحات:**

برای خواندن پیامک رسیده از این تابع استفاده می شود.

➢ **تابع سوم خواندن پیامک رسیده**

R_SMS3(PDU: WordBool; Indx: Integer; var SMS_Center, SMS_Type, Tel_Number,Time_stamp, S_Discharge, SMS_Text, TP_PID, TP_DCS, TP_VP, TP_ST, S_Reference: WideString): WideString;

➢ پارامترها :

PDU ← جهت خواندن نوع پیامک (PDU یا Text) مورد استفاده قرار می گیر و در حال حاضر فقط True می باشد.

- نوع پارامتر ورودی است

Indx ← شماره پیامک

- نوع پارامتر ورودی است

SMS_Center←شماره مرکز پیام ، پیامک خوانده شده.

- نوع پارامتر خروجی است

SMS_Type← نوع پیامک رسیده که شامل موارد ذیل می باشد:

- ✓ SMS RECEIVED: پیامک رسیده
- ✓ SMS STATUS REPORT: تاییدیه پیامک(Delivery)
- ✓ SMS SUBMIT: پیامک از تلفن همراه به مرکز پیام
- ✓ EMS RECEIVED(خدمات پیام رسانی پیشرفته) : پیامک پیشرفته دریافت شده
- ✓ EMS SUBMIT: پیامک پیشرفته از تلفن همراه به مرکز پیام

- نوع پارامتر خروجی است

Tel_Number← شماره تلفن پیامک ارسالی

- نوع پارامتر خروجی است.

Time_stamp ← تاریخ و ساعت دریافت پیامک توسط اپراتور تلفن همراه.

- نوع پارامتر خروجی است.

S_Discharge ← تاریخ و ساعت ارسال پیامک توسط اپراتور تلفن همراه.

- نوع پارامتر خروجی است.

SMS_Text ← متن پیامک

- نوع پارامتر خروجی است.

TP_PID ←

- نوع پارامتر خروجی است

TP_DCS ←

- نوع پارامتر خروجی است

TP_VP ←

- نوع پارامتر خروجی است

TP_ST ← نتیجه ارسال پیامک

Case TP_ST of

0x00: Short message delivered successfully
0x01: Forwarded, but status unknown
0x02: Replaced
0x20: Congestion, still trying
0x21: Recipient busy, still trying
0x22: No response recipient, still trying
0x23: Service rejected, still trying
0x24: QOS not available, still trying
0x25: Recipient error, still trying
0x40: RPC Error
0x41: Incompatible destination
0x42: Connection rejected
0x43: Not obtainable
0x44: QOS not available
0x45: No internetworking available
0x46: Message expired

0x47: Message deleted by sender
0x48: Message deleted by SMSC
0x49: Does not exist

- نوع پارامتر خروجی است

S_Reference ← شماره مرجعی است که جهت تشخیص اینکه این Delivery برای کدام پیامک ارسالی است این شماره مرجع برابر شماره مرجع هنگام ارسال پیامک باشد این Delivery مربوط به آن پیامک ارسالی می باشد.

- نوع پارامتر خروجی است

➢ **مقدار بازگشتی:**

رشته کاراکتری ← در صورتی که پیامک با موفقیت خوانده شود مقدار برگشتی یک رشته شامل یکسری کاراکتر عددی می باشد که هر چهار رقم آن با ویرگول از هم جدا شدند این اعداد چهار رقمی کد دوبایتی یک کاراکتر پیامک می باشد ، به عنوان مثال عدد هگز 0X06F1 کد دو کاراکتری عدد ۱ فارسی است.

➢ **توضیحات:**

برای خواندن پیامک رسیده از این تابع استفاده می شود.

➢ **تابع ارسال پیامک**

Send_SMS_PDU(const Country_Code, Phone_NO, SMS_Text: WideString): WordBool;

- پارامترها :

Country_Code ← کد کشور

- نوع پارامتر ورودی است

Phone_NO ← شماره تلفن

نکته: برای ارسال National یا داخلی شماره همراه با ۰ گرفته می شود مانند ۰۹۱۲۴۳۳۱۰۵۹ و برای ارسال International شماره بدون ۰ و با کد کشور گرفته می شود مانند ۹۸۹۱۲۴۳۳۱۰۵۹ ارسال می شود.

- نوع پارامتر ورودی است

SMS_Text ← متن پیامک فارسی یا هر زبان بین المللی دیگر

- نوع پارامتر ورودی است

- **مقدار بازگشتی:**

عدد منطقی ← اگر ارسال پیامک با موفقیت انجام شود True و در غیر اینصورت False بر میگرداند.

- **توضیحات:**

از این تابع برای ارسال پیامک فارسی یا هر زبان بین المللی دیگر مورد استفاده قرار می گیرد.

➢ **تابع دوم ارسال پیامک**

Send_SMS_PDU2(const Country_Code, Phone_NO, SMS_Text: WideString): WideString;

➢ پارامترها :

Country_Code ← کد کشور

- نوع پارامتر ورودی است

Phone_NO ← شماره تلفن

نکته: برای ارسال National یا داخلی شماره همراه با ۰ گرفته می شود مانند ۰۹۱۲۴۳۳۱۰۵۹ و برای ارسال International شماره بدون ۰ و با کد کشور گرفته می شود مانند ۹۸۹۱۲۴۳۳۱۰۵۹ ارسال می شود.

- نوع پارامتر ورودی است

SMS_Text ← متن پیامک فارسی یا هر زبان بین المللی دیگر

- نوع پارامتر ورودی است

➢ **مقدار بازگشتی:**

رشته کاراکتری ← اگر ارسال پیامک با موفقیت انجام شود یک عدد Reference به عنوان خروجی بر می گرداند این عدد برای Delivery مورد استفاده قرار می گیرد برای چک کردن اینکه کدام پیامک Delivery کدام پیامک ارسال است و اگر ارسال پیامک با خطا مواجه شود خروجی Null بر می گرداند.

- **توضیحات:**

از این تابع برای ارسال پیامک فارسی یا هر زبان بین المللی دیگر با خروجی Reference مورد استفاده قرار می گیرد.

- **تابع خواندن میزان شارژ باتری**

SMS_Battery(var Percent, Voltage: Integer): WordBool;

- پارامترها :

Percent ← در صد شارژ باتری(عدد بین ۰٪ تا ۱۰۰٪)

- نوع پارامتر خروجی است.

Voltage ← ولتاژ باتری(بین ۳۳۰۰ تا ۴۳۰۰ ولت)

- نوع پارامتر خروجی است.

- **مقدار بازگشتی:**

عدد منطقی ← اگر خواندن میزان ولتاژ با موفقیت انجام گیرد خروجی تابع True و در غیر اینصورت False خواهد بود.

- **توضیحات:**

از این تابع برای خواندن درصد شارژ باتری و میزان ولتاژ آن مورد استفاده قرار می گیرد.

درصد شارژ از صفر تا ۱۰۰ درصد و ولتاژ باتری از ۳۳۰۰ تا ۴۳۰۰ میلی ولت خواهد بود.

- **تابع وضعیت شارژ شدن باتری**

SMS_Bat_charging(var Status: Smallint): WordBool;

- پارامترها :

Status ← وضعیت شارژ شدن

۰: شارژ نمی شود.

۱: در حال شارژ می باشد.

- نوع پارامتر خروجی است.

- **مقدار بازگشتی:**

عدد منطقی ← اگر این تابع با موفقیت خوانده شود True بر می گرداند در غیر اینصورت False بر می گرداند.

- **تابع برقرای تماس**

GSM_Dial_NO(const CountryCode, Phone_NO: WideString; var Res_MSG: WideString): WordBool;

- پارامترها :

CountryCode ← کد کشور

- نوع پارامتر ورودی است.

Phone_NO ← شماره تلفن تماس

- نوع پارامتر ورودی است.

نکته: برای ارسال National یا داخلی شماره همراه با ۰ گرفته می شود مانند ۰۹۱۲۴۳۳۱۰۵۹ و برای ارسال International شماره بدون ۰ و با کد کشور گرفته می شود مانند ۹۸۹۱۲۴۳۳۱۰۵۹ ارسال می شود.

Res_MSG ← پیغام مودم

- نوع پارامتر خروجی است.

- **مقدار بازگشتی:**

عدد منطقی ← اگر فرمان گرفتن تماس برای مودم با موفقیت ارسال شود True و در غیر اینصورت False برمی گرداند.

- **توضیحات:**

از این تابع برای برقراری تماس مورد استفاده قرار می گیرد.

- **تابع ارسال پیامک Text**

```
Send_Text_SMS(PDU: WordBool; const Country_Code: WideString;
Const Phone_NO: WideString; const SMS_TXT: WideString): WordBool);
```

- **پارامترها :**

PDU ← مقدارش باید False قرار گیرد.

- نوع پارامتر ورودی است.

Country_Code ← کد کشور

- نوع پارامتر ورودی است.

Phone_NO ← شماره تلفن

- نوع پارامتر ورودی است.

نکته: برای ارسال National یا داخلی شماره همراه با ۰ گرفته می شود مانند ۰۹۱۲۴۳۳۱۰۵۹ و برای ارسال International شماره بدون ۰ و با کد کشور گرفته می شود مانند ۹۸۹۱۲۴۳۳۱۰۵۹ ارسال می شود.

SMS_Text ← متن پیامک انگلیسی

- نوع پارامتر ورودی است

➢ **مقدار بازگشتی:**

عدد منطقی ← اگر ارسال پیامک با موفقیت انجام شود True و در غیر اینصورت False بر میگرداند.

➢ **توضیحات:**

از این تابع برای ارسال پیامک فارسی یا هر زبان بین المللی دیگر مورد استفاده قرار می گیرد.

➢ **تابع ارسال ارسال AT Command**

GSM_Send_AT(const AT: WideString; var Response_Len: Integer): WideString

➢ **پارامترها :**

AT ← دستور AT

- نوع پارامتر ورودی است.

Response_Len ← طول پاسخ دستور AT از سمت GSM.

- نوع پارامتر خروجی است.

➢ **مقدار بازگشتی:**

رشته کاراکتری ← پاسخ دستور AT از سمت GSM.

➢ **توضیحات:**

از این تابع برای ارسال دستورات AT یا AT Command استفاده می شود.

- **تابع ارسال ارسال USSD**

function GSM_Send_USSD(const S_USSD: WideString): WideString;

- **پارامترها :**

S_USSD ← دستور USSD

مثال : #۱۱*۴۰*

- نوع پارامتر ورودی است.

- **مقدار بازگشتی:**

رشته کاراکتری ← نتیجه ارسال دستور USSD (پاسخ دستور USSD نیست!!!)

- **توضیحات:**

از این تابع برای ارسال دستورات USSD استفاده می شود.

نکته: برای استفاده از USSD نوع کاراکتر توسط تابع SMS_Char_Mode به GSM تنظیم شود.

- **تابع ارسال دریافت USSD**

function TMS_SMS_BAN.GSM_GetUSSD: WideString;

- **پارامترها :**

-

- **مقدار بازگشتی:**

رشته کاراکتری ← پاسخ دستور USSD .

- **توضیحات:**

از این تابع برای دریافت نتیجه دستور ارسال شده USSD استفاده می شود.

- **تابع تنظیم شماره دستگاه**

GSM_Set_Device_NO(D_NO: Smallint): WordBool;

- **پارامترها :**

D_NO ← شماره دستگاه

- نوع پارامتر ورودی است.

- **مقدار بازگشتی:**

عدد منطقی ← اگر با موفقیت انجام شود True و در غیر اینصورت False بر میگرداند.

- **توضیحات:**

از این تابع برای تنظیم شماره دستگاه مورد استفاده قرار می گیرد برای اینکه چند دستگاه که به یک سیستم متصل می شوند قابل تمیز دادن از هم شوند از طریق تنظیم شماره دستگاه با استفاده از این تابع امکان پذیر می باشد.

- **تابع ارسال ارسال AT Command**

```
GSM_Get_Device_NO: Smallint;
```

- **پارامترها :**

-

- **مقدار بازگشتی:**

عدد صحیح ← شماره دستگاه

- **توضیحات:**

از این تابع برای خواندن شماره دستگاه که توسط تابع قبلی تنظیم شده بود، مورد استفاده قرار می گیرد با خواندن شماره دستگاه با استفاده از این تابع می توان چند دستگاهی که به یک سیستم متصل می شوند را تشخیص داد.

➢ نکات مهم:

- برای ارسال Enter از کاراکتر 10# استفاده می شود.
- برای ارسال پیامک انگلیسی با تابع Send_Text_SMS اگر وضعیت ارسال پیامک در حالت PDU باشد ابتدا باید تنظیم نوع کاراکتر بر روی GSM انتخاب شود سپس با تابع SMS_PDU_Mode نوع پیامک را به پیامک متن انگلیسی تغییر می دهیم. در این حالت فقط تابع Send_Text_SMS کار خواهد کرد، برای برگشتن به حالت پیامک PDU یعنی ارسال پیامک با تابع Send_SMS_PDU نوع کاراکتر را به UCS2 تغییر می دهیم سپس نوع پیامک را از طریق تابع SMS_PDU_Mode به پیامک فارسی PDU تغییر می دهیم.